LoRa Gateway_WIFI Data Protocol of TCP Communication

Document Revision History

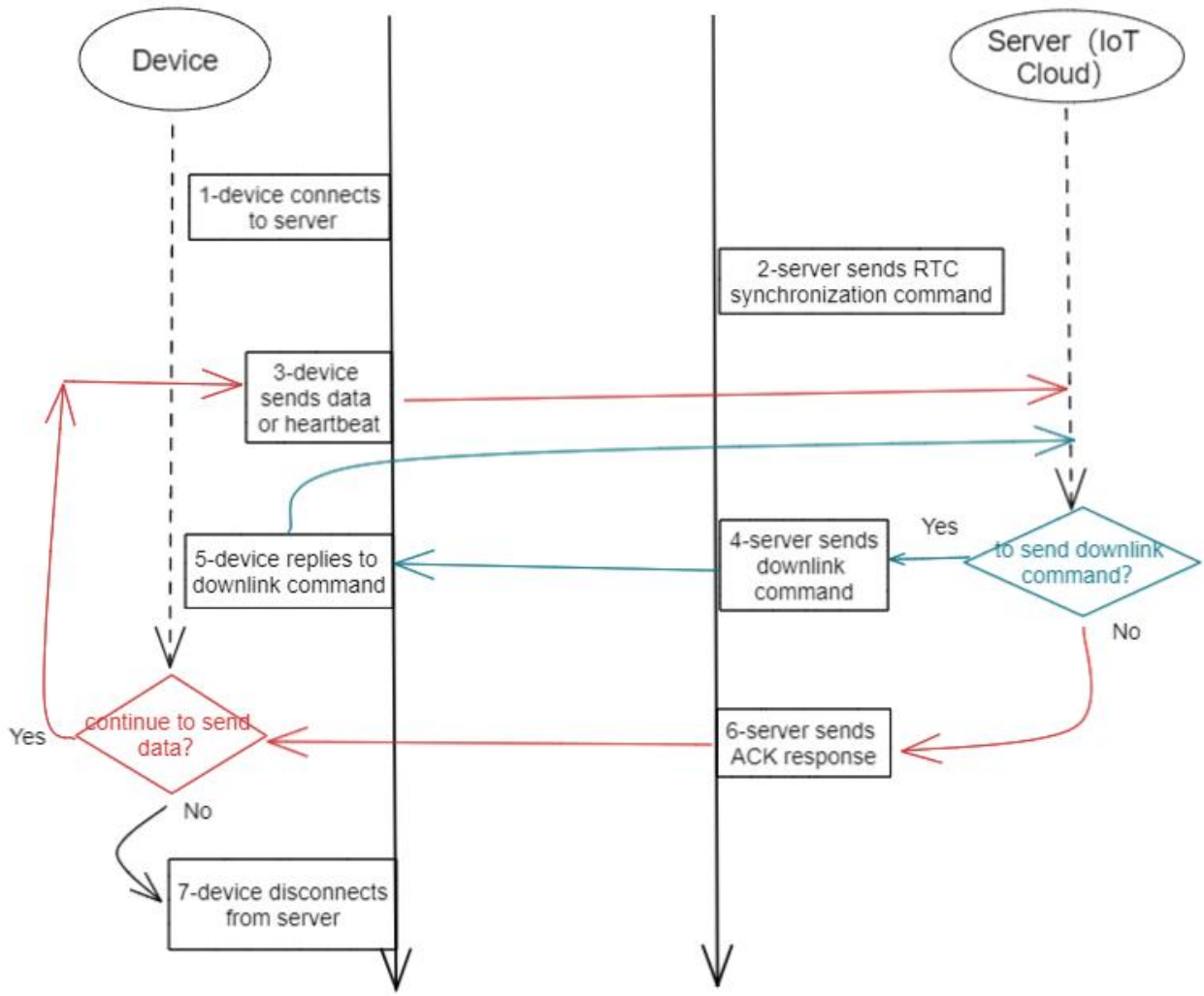
Date	Author	Version	Description
2025/8/6	Michael,Lisa	V2.0	New document structure and more detailed presentation with tables and images.

Table of Contents

1. The Interaction Process and Related Requests or Replies 3
1.1 The Interaction Process Between the Server and the Device 3
1.2 Detailed Description of the Interaction Process 3
1.3 The Basic Format of the Server Requests or Replies 4
1.4 A List of Server Requests or Replies 4
1.5 Basic Format of the Device Requests or Replies 5
1.6 List of Device Requests or Replies 6
2. Detailed Description of All Requests or Replies Between the Device and the Server 7
2.1 Server RTC Synchronization Command 7
2.2 Device Basic Data Sending 7
2.3 Server ACK Response 12
2.4 Server Downlink Command 12
2.5 Device's General Reply to Downlink Command 13
2.6 Device's Reply to 801 Downlink Command 14
2.7 Server 900 Downlink Command 15
2.8 Device's Reply to 900 Downlink Command 16
2.9 Checksum Calculation 17

1. The Interaction Process and Related Requests or Replies

1.1 The Interaction Process Between the Server and the Device



1.2 Detailed Description of the Interaction Process

The following is based on a typical server-side implementation. The actual server-side can choose to implement all or part of the functionality without violating the device-side logic.

1. "1-Device connects to server": When the reporting time interval is reached or triggered by other events, the gateway device connects to server.
2. "2-Server sends RTC synchronization command ": After being connected by the device, the server can send a command to synchronize the device's RTC time. The device does not need to wait for the server's RTC synchronization command to send data or heartbeat.
3. "3-Device sends data or heartbeat": After the gateway device connects to the server, it starts to report basic data or heartbeat data to the server(Gateway have not heartbeat data).
4. The server judges the condition of "whether to send downlink command". After receiving and processing the sent

data, the server checks the downlink command queue to determine "whether to send downlink command". If yes, execute "4-Server sends downlink command"; if no, execute "6-Server sends ACK reply".

5. "4-Server sends downlink command": The server takes the command at the head of the downlink command queue and sends it to the device. These commands include the following 2 types:
   - Server downlink commands.
   - Server 900 downlink commands.
6. "5-Device replies to downlink command": After receiving and executing the command, the device returns the corresponding reply to the server. These replies include the following 3 types:
   - Device's general reply to downlink command.
   - Device's reply to 801 downlink command.
   - Device's reply to 900 downlink command.
7. The server continues to judge the condition of "whether to send downlink command". If the judgment is yes, it cyclically executes steps 4-6 above. Exit when the judgment is no.
8. "6-Server sends ACK reply": The server indicates that there is no commands to issue by sending ACK response, and hands over control to the device.
9. The device judges the condition of "continue to send data". The device checks the data-to-send queue to determine whether "continue to send data".
   - If yes, repeat "3-Device sends data or heartbeat". Then it may enter the sub-loop of steps 4-6 above. After exiting the sub-loop, then to judge whether "continue to send data".
   - If no, execute "7-Device disconnects from server".
10. "7-Device disconnects from server". The device actively disconnects from the server, ending the communication between two sides. The server needs to wait for the next connection from the device to communicate. Therefore, when the server needs to send a downlink command to the device, it cannot issue it immediately and can only store it in the downlink command queue first.

1.3 The Basic Format of the Server Requests or Replies

It is ASCII string in the following format: @{keyword}, {RE_format} #\r\n

The RE_format of the server's downlink commands is variable, and the RE_format of other requests or replies are fixed, please refer to the table in the next section

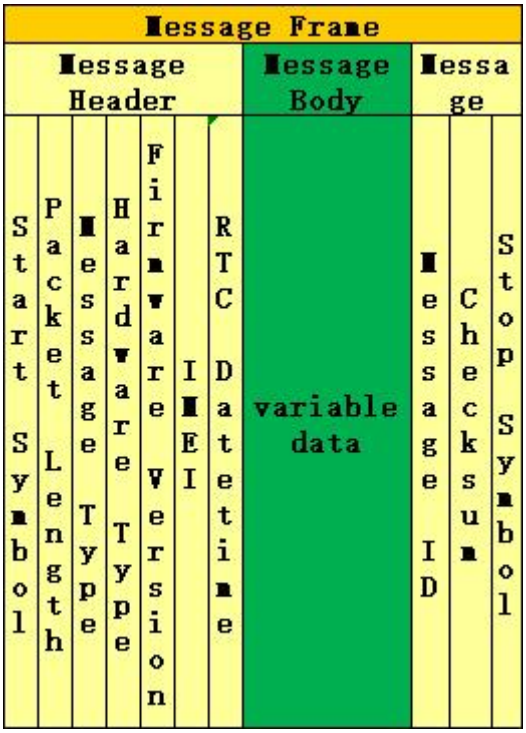
1.4 A List of Server Requests or Replies

NO.	Requests or Replies	Keyword	RE_format	Examples
1	Server RTC synchronization	UTC	UTC time in the format: yyyy-MM-dd HH:mm:ss	@UTC,2021-11-24 02:56:43#\r\n

	command			
2	Server ACK reply	ACK	Decimal message serial number (message ID), range[1,9999], with variable length	@ACK,1035#\r\n
3	Server downlink command, for setting or reading gateway device	CMD	Related to specific commands, see detailed description later	@CMD,*000000,018,10#,#\r\n
4	Server 900 downlink command, for setting or reading tags through gateway device	CMD	Including 900 instruction and related to specific sub-instruction, see detailed description later	@CMD,*000000,900,12345678,002,5#,#\r\n

1.5 Basic Format of the Device Requests or Replies

The message packet of device requests or replies is a byte string, different from the character string of server packet. The basic composition format of the byte string message body is: message header + message body + message tail



Note:

- For different requests or replies, the lengths of the message header and message tail are fixed, but the contents may vary.
- For different requests or replies, the length and content of the message content are different. See the detailed description sections for details.

Data block	Number of bytes	Content	Description
Start symbol	2	'TZ'	Header of every packet
Packet length	2	variable	The length from message type to Checksum
Message Type	2	'$$'	'$$' means the message type is basic data sending
Hardware type	2	04H 06H	0406 for LoRa gateway
Firmware version	4	variable	i.e. 02H 01H 00H 00H means Firmware version is 2.1
IMEI	8	variable	BCD format. the 4 high bits of the first byte should be ignored and each 4 bits of the others represent a character. then a 15-digit number will be got. i.e.06H 51H 88H 49H 07H 90H 00H 03H means IMEI is 651884907900003
RTC time	6	variable	The RTC time when this message is packed. The sequence is Year Month Day Hour Minute Second i.e. 12H 0CH 0AH 05H 37H 13H means 2018/12/10/ 05：55：19
Message Body	X	variable	Variable data
Message ID	2	variable	The value range of this part is between 1 and 9999
Checksum	2	variable	The CRC16 checksum of bytes from message type to Message ID.
Stop Symbol	2	0DH 0AH	Indicate this frame/packet is finished

1.6 List of Device Requests or Replies

NO.	Requests or Replies	Description	Data format
1	Basic data sending request	Device status data, tag data records, etc.	Basic data sending request format, please see the detailed explanation below
2	Device's general reply to downlink command	General reply to downlink commands which are mainly related to the gateway device itself (except for instruction 801/900). For reading/query command, reply with query result; For setting command, reply with setting results	reply to downlink commands format,please see the detailed explanation below
3	Device's reply to 801 downlink command	Special reply to 801 downlink command.	reply to 801 downlink command format,please see the detailed explanation below
4	Device's reply to 900 downlink command	Special reply to 900 downlink command which is related to the tags.	reply to 900 downlink command,format,please see the detailed explanation below

2. Detailed Description of All Requests or Replies Between the Device and the Server

2.1 Server RTC Synchronization Command

Request format: @UTC,yyyy-MM-dd HH:mm:ss#

Example: @UTC,2021-11-24 02:56:43#

Note: It must be set to UTC time. And it is recommended that the server sets the RTC time each time when the device connects to the server; otherwise, the RTC time may be incorrect.

2.2 Device Basic Data Sending

2.2.1 Data Blocks of Message Frame

Data Sending																									
Message Header							Message Body																Message Tail		
								Device Data							Tag Data										
Start Symbol	Packet Length	Message Type	Hardware Type	Firmware Version	IMEI	RTC Datetime	Reserved1	Device Data Length	Data Type	Device Status	Reserved2	Battery Voltage	Power Voltage	Reserved3	Tag Data Length	Tag Type	Number of Records	One Record Length	Tag Data Records	Reserved4	Reserved5	Message ID	Checksum	Stop Symbol
2	2	2	2	4	8	6	2	2	1	1	2	2	2	A	2	1	1	1	X	B	C	2	2	2

2.4.2 Detailed Description for All Data Blocks

Data block	Number of bytes	Content	Description
Start symbol	2	'TZ'	Header of every packet
Packet length	2	variable	The length from message type to Checksum
Message Type	2	'$$'	'$$' means the message type is basic data sending
Hardware type	2	04H 06H	0406 for LoRa gateway

Firmware version	4	variable	i.e. 02H 01H 00H 00H means Firmware version is 2.01
IMEI	8	variable	BCD format. the 4 high bits of the first byte should be ignored and each 4 bits of the others represent a character. then a 15-digit number will be got. i.e.06H 51H 88H 49H 07H 90H 00H 03H means IMEI is 651884907900003
RTC time	6	variable	The RTC time when this message is packed. The sequence is Year Month Day Hour Minute Second i.e. 12H 0CH 0AH 05H 37H 13H means 2018/12/10/ 05：55：19
Reserved1	2	00H 00H	Reserved for future use. Currently, the content is 0
Device Data Length	2	variable	Data length of device data. 00H 00H indicates no such data
Data Type	1	variable	There are four types: AAH - normal; 10H - low voltage alarm; 60H – charging alarm; 61H - power failure alarm;
Device Status	1	variable	Bit7: 1 - power plugged in, 0 - power not plugged in; Bit6: 1 – last frame (no more data to send), 0 - not last frame (continue to send data); Bit5~0: not defined yet
Reserved2	2	00H 00H	Reserved for future use. Currently, the content is 0
Battery Voltage	2	variable	Unit:10mv, MSB first i.e. 01H A8H=424, 424*10=4.24V
Power Voltage	2	variable	Unit:10mv, MSB first i.e. 04H DFH=1247, 1247*10=12.47V
Reserved3	A=0	NC	Reserved for future use. Currently, no bytes is used.
Tag Data Length	2	variable	Length of tag data. 00H 00H indicates no such data
Tag Type	1	variable	01 - Tag07/07B/08/08L/09 04 - Tag08B 08 - Tag11
Number of Records	1	variable	Indicates number of tag data records in the subsequent Tag Data Records
One Record Length	1	variable	Length of one single tag data record. For different tag type: 01 - 17 bytes (humidity data block has 1 byte, unit is 1%); 04 - 18 bytes (default humidity data block has 2 bytes, unit is 0.1%); 08 - 19 bytes
Tag Data Records	X	variable	Composed of multiple tag data records. For different Tag type: 01 or 04 - Tag07/07B/08/08B/08L/09: the format of one single tag data record is like: ID + status + battery voltage + temperature + humidity +

			RSSI +RTC time; 08 - Tag11: the format of one single tag data record is like: ID + status + battery voltage + voltage value + current value + switch value + RSSI + RTC time; X = (Number of Records) * (One Record Length); For details, refer to "Detailed Description for one single Tag Data Record"
Reserved4	B=0	NC	Reserved for future use. Currently, no bytes is used.
Reserved5	C=0	NC	Reserved for future use. Currently, no bytes is used.
Message ID	2	variable	The value range of this part is between 1 and 9999
Checksum	2	variable	The CRC16 checksum of bytes from message type to Message ID.
Stop Symbol	2	0DH 0AH	Indicate this frame/packet is finished

2.2.3 Detailed Description for All Data Blocks of One Single Tag Data Record

Data block	Number of bytes	Tag Type：01 or 04	Tag Type：08
Tag Data Record	For tag type： 01 - 17 04 - 18 08 - 19	01-Tag07/07B/08/08B/08L/09 04-Tag08B Format: ID + status + battery voltage + [temperature + humidity] + RSSI + RTC time	08-Tag11 Format: ID + status + battery voltage + [voltage value + current value + switch value] + RSSI + RTC time
ID	4	8 digits, e.g.: 72180476	8 digits, e.g.: 80000003
Status	1	Bit7: voltage status, 1 - undervoltage, 0 - normal voltage; Bit6: temperature alarm status, 1 - temperature alarm, 0 - normal temperature; Bit5: Tag button, 1 - Tag button pressed, 0 - not pressed; Bit4: gateway response required flag (set by 009 command), 1 - ACK enabled, 0 - ACK disabled; Bit3: RTC time flag, 1 - Tag RTC time enabled, 0 - Tag RTC time disabled; Bit2 - bit0: reserved	Bit7: voltage status, 1 - undervoltage, 0 - normal voltage; Bit6: alarm status, 1 - alarm, 0 - normal; Bit5: Tag button, 1 - Tag button pressed, 0 - not pressed; Bit4~0: reserved
Battery Voltage	2	Battery voltage: 2 bytes, unit 1mv, MSB first. e.g., 0EH 38H indicates voltage is	Battery voltage: 2 bytes, unit 1mv, MSB first. e.g., 0EH 24H indicates

		3.64V	voltage is 3.62V
Temperature	2	MSB first, unit 0.1℃. Bit15 indicates whether the Tag is working normally: 1 - abnormal, 0 - normal; Bit14 indicates temperature polarity: 1 - negative temperature, 0 - positive temperature; Bit13-Bit0 indicates temperature value. e.g., 00H D6H indicates temperature is 21.4℃; 40H D6H indicates temperature is -21.4℃; 80H 00H indicates Tag is working abnormally	NA
Humidity	For Tag Type: 01 - 1 04 - 2	For TAG07/07B/08/08L/09: 1 byte, unit: 1%; For TAG08B: 2 bytes, unit: 0.1%. Note: ● If the first byte is FF, it indicates that this tag has no humidity data; ● If it is 30H, it indicates humidity is 48%; ● If it is 02H CFH, it indicates humidity is 71.9%	NA
Voltage Value	2	NA	Unit mv, high bit first. e.g., 0FH 44H indicates voltage value is 3908mv
Current Value	2	NA	Unit uA, high bit first. e.g., 26H 3AH indicates current value is 9786uA
Switch Value	1	NA	00H - on, 01H - off
RSSI	1	RSSI: 1 byte, unit: -dbm. e.g., 07H indicates RSSI value is -7dbm	RSSI: 1 byte, unit: -dbm. e.g., 4AH indicates RSSI value is -74dbm
RTC Time	6	Time when tag data is recorded, each byte sequentially represents year, month, day, hour, minute, second. e.g.: 12H 0CH 0AH 05H 37H 12H indicates time is: 2018\12\10 05:55:18	Time when tag data is recorded, each byte sequentially represents year, month, day, hour, minute, second. e.g.: 16H 0CH 06H 02H 19H 2EH indicates time is:2022\12\06 02:25:46

2.2.4 Message Frame Samples

WIFI Gateway+TAG07B Data Example:

54 5A 00 3C 24 24 04 06 02 01 00 00 06 51 88 49 07 90 00 03 12 0C 0A 05 37 13 00 00 00 08 AA C0 00 00 01 A8 04 DF 00 14 01 01 11 72 18 04 76 20 0E 38 00 D6 30 07 12 0C 0A 05 37 12 03 F5 E8 1C 0D 0A

WIFI Gateway+TAG08B Data Example:

54 5A 00 3D 24 24 04 06 03 09 00 00 06 52 28 49 07 90 03 49 19 06 18 07 1F 36 00 00 00 08 AA C0 00 00 01 A7 04 CB 00 15 04 01 12 82 12 00 03 18 0D DE 40 E1 02 B0 58 19 06 18 07 1E 0B 00 05 06 24 0D 0A

WIFI Gateway+TAG011 Data Example:

54 5A 00 3E 24 24 04 06 03 05 00 00 06 52 28 49 07 90 03 50 16 0C 06 02 1A 01 00 00 00 08 AA C0 00 00 01 A8 04 DD 00 16 08 01 13 80 00 00 03 38 0E 24 0F 44 26 3A 01 4A 16 0C 06 02 19 2E 00 02 7E 5F 0D 0A

2.2.5 Detailed Description of Message Frame Samples by Each Data Block

Data block	Number of bytes	WIFI Gateway + TAG07B	WIFI Gateway + TAG08B	WIFI Gateway + TAG11
Start symbol	2	54 5A - 'TZ'	54 5A - 'TZ'	54 5A - 'TZ'
Packet length	2	00 3C - 60 bytes	00 3D - 61 bytes	00 3E - 64 bytes
Message Type	2	24 24 - '$$'	24 24 - '$$'	24 24 - '$$'
Hardware type	2	04 06	04 06	04 06
Firmware version	4	02 01 00 00 - 2.01	03 9F 00 00 - 3.09	03 05 00 00 - 3.05
IMEI	8	06 51 88 49 07 90 00 03 - 0651884907900003	06 52 28 49 07 90 03 49 - 06522849079003	06 52 28 49 07 90 03 50 - 652284907900350
RTC time	6	12 0C 0A 05 37 13 - 2018\12\10 05:55:19	19 06 18 07 1F 36 - 2025\06\24 07:31:54	16 0C 06 02 1A 01 - 2022\12\06 02:26:01
Reserved1	2	00 00	00 00	00 00
Device Data Length	2	00 08 - 8 bytes	00 08 - 8 bytes	00 08 - 8 bytes
Data Type	1	AA—Normal data	AA—Normal data	AA—Normal data
Device Status	1	C0 - connect to power, last packet	C0 - connect to power, last packet	C0 - connect to power, last packet
Reserved2	2	00 00	00 00	00 00
Battery Voltage	2	01 A8 - 4.24V	01 A7 - 4.23V	01 A8 - 4.24V
Power Voltage	2	04 DF - 12.47V	04 CB - 12.27V	04 DD - 12.45V
Tag Data Length	2	00 14 - 20 bytes	00 15 - 21 bytes	00 16 - 222 bytes
Tag Type	1	01	04	08
Number of Records	1	01	01	01
One Record Length	1	11 - 17	12 - 18	13 - 19
Tag Data Records	For tag type: 01 - 17	72 18 04 76 20 0E 38 00 D6 30 07 12 0C 0A 05 37 12	82 12 00 03 18 0D DE 40 E1 02 B0 58 19 06 18 07 1E 0B	80 00 00 03 38 0E 24 0F 44 26 3A 01 4A 16 0C 06 02 19 2E

	04 - 18 08 - 19	Number of Records:1 Data blocks of Record1: ID: 72180476 Status: 20 Battery voltage: 0E 38 - 3.64V; Temperature: 00 D6 - 21.4℃; Humidity: 30 - 48%; RSSI: 07 - 7dBm; RTC Time: 12 0C 0A 05 37 12 — 2018\12\10 05:55:18	Number of Records:1 Data blocks of Record1: ID: 82120003 Status: 18 Battery voltage: 0D DE - 3.55V; Temperature: 40 E1 - -22.5℃; Humidity: 02B0 - 68.8%; RSSI: 58 - 88dBm; RTC Time: 19 06 18 07 1E 0B — 2025\06\24 07:30:11	Number of Records:1 Data blocks of Record1: ID: 80 00 00 03 Status: 38 Battery voltage: 0E 24 - 3.62V; Voltage: 0F 44 - 3908mv; Current: 26 3A - 9786uA; Switch value: 01 - on; RSSI: 4A - 74dBm; RTC Time: 16 0C 06 02 19 2E— 2022\12\06 02:25:46
Message ID	2	03 F5 - 1013	00 05 - 0005	00 02 - 0002
Checksum	2	E8 1C	06 24	7E 5F
Stop Symbol	2	0D 0A	0D 0A	0D 0A

2.3 Server ACK Response

Reply format: @ACK,{message-ID#\r\n

Example: @ACK,1035#\r\n

Note: message-ID, the value range of this part is between 1 and 9999.

2.4 Server Downlink Command

Request format: @CMD,{CMD-format},#\r\

Example 1 (Set data transmission interval of gateway device): @CMD,*000000,018,10#,#\r\n

Example 2 (Get/Read data transmission interval of gateway device): @CMD,*000000,800,018#,#\r\n

Example 3 (Get version of WiFi module of gateway device): @CMD,*000000,801#,#

Note：

- For the CMD-format of specific instructions, please refer to the document "LoRa Gateway_WIFI User Manual (command part)", and note the "Format" column, which is equivalent to CMD-format.
- {CMD-format} varies according to different instructions, in the following format:
  {CMD-format}=*{password},{ins-code}[,{ins-paras}]#
  - {} indicates variables, [] indicates optional.
  - password: password for accessing the gateway device, default is 000000.
  - ins-code: instruction code like 018, 800, 801, etc.
  - ins-paras: parameters of the instruction. For different instructions, parameters number is 0 to many.
- Note 800 in the example 2. 800 is used to read value of any instruction when instruction code is used as parameter,

format like: @CMD,*000000,800,{ins-code}#,#\r\n

2.5 Device's General Reply to Downlink Command

2.5.1 Data Blocks of Message Frame

Reply to Downlink Command													
Message Header							Message Body				Message Tail		
							Command Result						
Start Symbol	Packet Length	Message Type	Hardware Type	Firmware Version	IMEI	RTC Datetime	Command Type	Instruction Code	Result Code	Retrieved Value	Message ID	Checksum	Stop Symbol
2	2	2	2	4	8	6	1	2	1	X	2	2	2

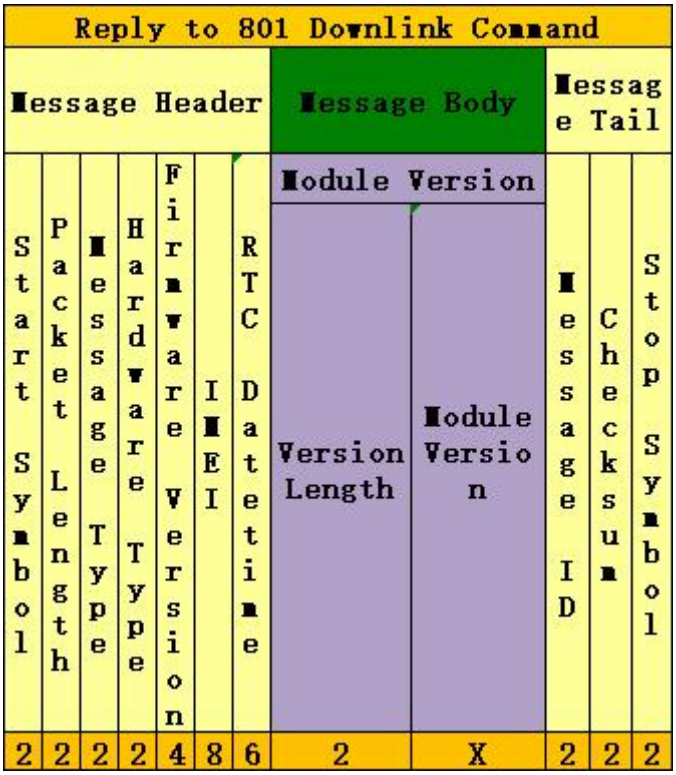
2.5.2 Detailed description for all data blocks

Data block	Number of bytes	Content	Description
Start symbol	2	'TZ'	This is the header of every packet
Packet length	2	variable	The length from Message Type to Checksum
Message Type	2	'$D'	'$D' means the message type is general reply to downlink Command
Hardware type	2	04H 06H	0406 for LoRa gateway
Firmware version	4	variable	i.e. 02H 01H 00H 00H means Firmware version is 2.01
IMEI	8	variable	BCD format. the 4 high bits of the first byte should be ignored and each 4 bits of the others represent a character. then a 15-digit number will be got. i.e.06H 51H 88H 49H 07H 90H 00H 03H means IMEI is 651884907900003
RTC time	6	variable	The RTC time when this message is packed.

			The sequence is Year Month Day Hour Minute Second i.e. 12H 0CH 0AH 05H 37H 13H means 2018/12/10/ 05：55：19
Command Type	1	variable	The type of downlink command to reply. 00H means setting command, 01H means reading command
Instruction Code	2	variable	The last instruction code in server's command.
Result Code	1	variable	Result code of setting or reading command: 00H indicates success, 05H indicates the command is not supported, 08H indicates failure, 09H indicates the device cannot be set now
Retrieved Value	X	variable	For setting command, there is no such data, byte number is 0; for reading command, it is the retrieved value. This data block is in ASCII format, and the data content is separated by commas for different fields. For details, see the command/instruction table. For example, when reading IP, this data is 1,g-cloud.tzonedigital.com,18801
Message ID	2	variable	The value range of this part is between 1 and 9999
Checksum	2	variable	The CRC16 checksum of bytes from message type to Message ID.
Stop Symbol	2	0DH 0AH	Indicate this frame/packet is finished

2.6 Device's Reply to 801 Downlink Command

2.6.1 Data Blocks of Message Frame



2.6.2 Detailed description for all data blocks

Data block	Number of bytes	Content	Description
Start symbol	2	'TZ'	This is the header of every packet
Packet length	2	variable	The length from message type to Checksum
Message Type	2	'$M'	'$M' means the message type is reply to 801 downlink Command
Hardware type	2	04H 06H	0406 for LoRa gateway
Firmware version	4	variable	i.e. 02H 01H 00H 00H means Firmware version is 2.01
IMEI	8	variable	BCD format. the 4 high bits of the first byte should be ignored and each 4 bits of the others represent a character. then a 15-digit number will be got. i.e.06H 51H 88H 49H 07H 90H 00H 03H means IMEI is 651884907900003
RTC time	6	variable	The RTC time when this message is packed. The sequence is Year Month Day Hour Minute Second i.e. 12H 0CH 0AH 05H 37H 13H means 2018/12/10/ 05：55：19
Version Length	2	variable	00H 00H indicates no such data
Module Version	X	variable	ASCII format
Message ID	2	variable	The value range of this part is between 1 and 9999
Checksum	2	variable	The CRC16 checksum of bytes from message type to Message ID.
Stop Symbol	2	0DH 0AH	Indicate this frame/packet is finished

2.7 Server 900 Downlink Command

Request format of 900 downlink command: @CMD, *{password},900,{ID},{subCMD-format}#,#\r\

Example 1 (Set TAG data transmission interval): @CMD,*000000,900,12345678,002,5#,#

Example 2 (Get/Read TAG data transmission interval): @CMD,*000000,900,12345678,253,002#,#

Note:

- {} indicates variables.
- password: Password for accessing the gateway device, default is 000000.
- ID: device ID of ta, 8 characters.
- subCMD-format: Varies according to different sub-instructions. For details, please refer to the document "Description of LoRa Gateway Issuing Commands to Synchronize TAG Configuration.docx", and note the "instruction" and "Format" columns in the table. The former is sub-instruction, and the latter is equivalent to subCMD-format.
- Note 253 in the example 2. 253 is used to read value of any sub-instruction for TAG when sub-instruction code is

used as parameter, format like: @CMD,*000000,900,12345678,253,{sub-ins-code}#,#

2.8 Device's Reply to 900 Downlink Command

2.8.1 Data Blocks of Message Frame

Reply to 900 Downlink Command														
Message Header							Message Body					Message Tail		
							Command Result							
Start Symbol	Packet Length	Message Type	Hardware Type	Firmware Version	IMEI	RTC Datetime	Command Type	Tag ID	Instruction Code	Result Code	Retrieved Value	Message ID	Checksum	Stop Symbol
2	2	2	2	4	8	6	1	4	2	1	X	2	2	2

2.8.2 Detailed Description for All Data Blocks

Data block	Number of bytes	Content	Description
Start symbol	2	'TZ'	This is the header of every packet
Packet length	2	variable	The length from message type to Checksum
Message Type	2	'$T'	'$T' means the message type is reply to 900 downlink command
Hardware type	2	04H 06H	0406 for LoRa gateway
Firmware version	4	variable	i.e. 02H 01H 00H 00H means Firmware version is 2.01
IMEI	8	variable	BCD format. the 4 high bits of the first byte should be ignored and each 4 bits of the others represent a character. then a 15-digit number will be got. i.e.06H 51H 88H 49H 07H 90H 00H 03H means IMEI is 651884907900003

RTC time	6	variable	The RTC time when this message is packed. The sequence is Year Month Day Hour Minute Second i.e. 12H 0CH 0AH 05H 37H 13H means 2018/12/10/ 05：55：19
Command Type	1	variable	The command type of downlink command to reply. 00H means setting Command, 01H means reading command
Tag ID	4	variable	Device ID of tag
Instruction Code	2	variable	The last instruction code in server's command.
Result Code	1	variable	Result code of setting or reading command: 00H indicates success, 05H indicates the command is not supported, 08H indicates failure, 09H indicates the device cannot be set now
Retrieved Value	X	variable	For setting, there is no such data, byte number is 0; for reading, it is the retrieved value. This data block is in ASCII format, and the data content is separated by commas for different fields. For details, see the command/instruction table. For example, when reading IP, the data is 1,g-cloud.tzonedigital.com,18801
Message ID	2	variable	The value range of this part is between 1 and 9999
Checksum	2	variable	The CRC16 checksum of bytes from message type to Message ID.
Stop Symbol	2	0DH 0AH	Indicate this frame/packet is finished

2.9 Checksum Calculation

Checksum Calculation

```
onst unsigned char auchCRCHi[] =
{ 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0,
0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01,
        0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00,
0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01,
        0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00,
0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00,
        0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01,
0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01,
        0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00,
0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00,
        0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01,
0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00,
        0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01,
0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01,
        0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00,
0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40 };

const unsigned char auchCRCLo[] =
```

```
{ 0x00, 0xC0, 0xC1, 0x01, 0xC3, 0x03, 0x02, 0xC2, 0xC6, 0x06, 0x07, 0xC7, 0x05, 0xC5, 0xC4, 0x04, 0xCC,
0x0C, 0x0D, 0xCD, 0x0F, 0xCF, 0xCE, 0x0E, 0x0A, 0xCA, 0xCB, 0x0B, 0xC9, 0x09, 0x08, 0xC8, 0xD8,
        0x18, 0x19, 0xD9, 0x1B, 0xDB, 0xDA, 0x1A, 0x1E, 0xDE, 0xDF, 0x1F, 0xDD, 0x1D, 0x1C, 0xDC,
0x14, 0xD4, 0xD5, 0x15, 0xD7, 0x17, 0x16, 0xD6, 0xD2, 0x12, 0x13, 0xD3, 0x11, 0xD1, 0xD0, 0x10, 0xF0,
        0x30, 0x31, 0xF1, 0x33, 0xF3, 0xF2, 0x32, 0x36, 0xF6, 0xF7, 0x37, 0xF5, 0x35, 0x34, 0xF4, 0x3C,
0xFC, 0xFD, 0x3D, 0xFF, 0x3F, 0x3E, 0xFE, 0xFA, 0x3A, 0x3B, 0xFB, 0x39, 0xF9, 0xF8, 0x38, 0x28,
        0xE8, 0xE9, 0x29, 0xEB, 0x2B, 0x2A, 0xEA, 0xEE, 0x2E, 0x2F, 0xEF, 0x2D, 0xED, 0xEC, 0x2C, 0xE4,
0x24, 0x25, 0xE5, 0x27, 0xE7, 0xE6, 0x26, 0x22, 0xE2, 0xE3, 0x23, 0xE1, 0x21, 0x20, 0xE0, 0xA0,
        0x60, 0x61, 0xA1, 0x63, 0xA3, 0xA2, 0x62, 0x66, 0xA6, 0xA7, 0x67, 0xA5, 0x65, 0x64, 0xA4, 0x6C,
0xAC, 0xAD, 0x6D, 0xAF, 0x6F, 0x6E, 0xAE, 0xAA, 0x6A, 0x6B, 0xAB, 0x69, 0xA9, 0xA8, 0x68, 0x78,
        0xB8, 0xB9, 0x79, 0xBB, 0x7B, 0x7A, 0xBA, 0xBE, 0x7E, 0x7F, 0xBF, 0x7D, 0xBD, 0xBC, 0x7C,
0xB4, 0x74, 0x75, 0xB5, 0x77, 0xB7, 0xB6, 0x76, 0x72, 0xB2, 0xB3, 0x73, 0xB1, 0x71, 0x70, 0xB0, 0x50,
        0x90, 0x91, 0x51, 0x93, 0x53, 0x52, 0x92, 0x96, 0x56, 0x57, 0x97, 0x55, 0x95, 0x94, 0x54, 0x9C, 0x5C,
0x5D, 0x9D, 0x5F, 0x9F, 0x9E, 0x5E, 0x5A, 0x9A, 0x9B, 0x5B, 0x99, 0x59, 0x58, 0x98, 0x88,
        0x48, 0x49, 0x89, 0x4B, 0x8B, 0x8A, 0x4A, 0x4E, 0x8E, 0x8F, 0x4F, 0x8D, 0x4D, 0x4C, 0x8C, 0x44,
0x84, 0x85, 0x45, 0x87, 0x47, 0x46, 0x86, 0x82, 0x42, 0x43, 0x83, 0x41, 0x81, 0x80, 0x40 };


unsigned int CRC16(unsigned char *puchMsg, unsigned int DataLen)
{
    unsigned char uchCRCHi = 0xFF;
    unsigned char uchCRCLo = 0xFF;
    unsigned char *pt;
    unsigned int Len;
    unsigned char uIndex;

    pt = puchMsg;
    Len = DataLen;

    while(Len--)
    {
        uIndex = uchCRCLo ^ (*pt++);
        uchCRCLo = uchCRCHi ^ (auchCRCHi[uIndex]);
        uchCRCHi = auchCRCLo[uIndex];
    }

    return (uchCRCHi << 8 | uchCRCLo);
}
```